Algoritmia
Presentación de prácticas

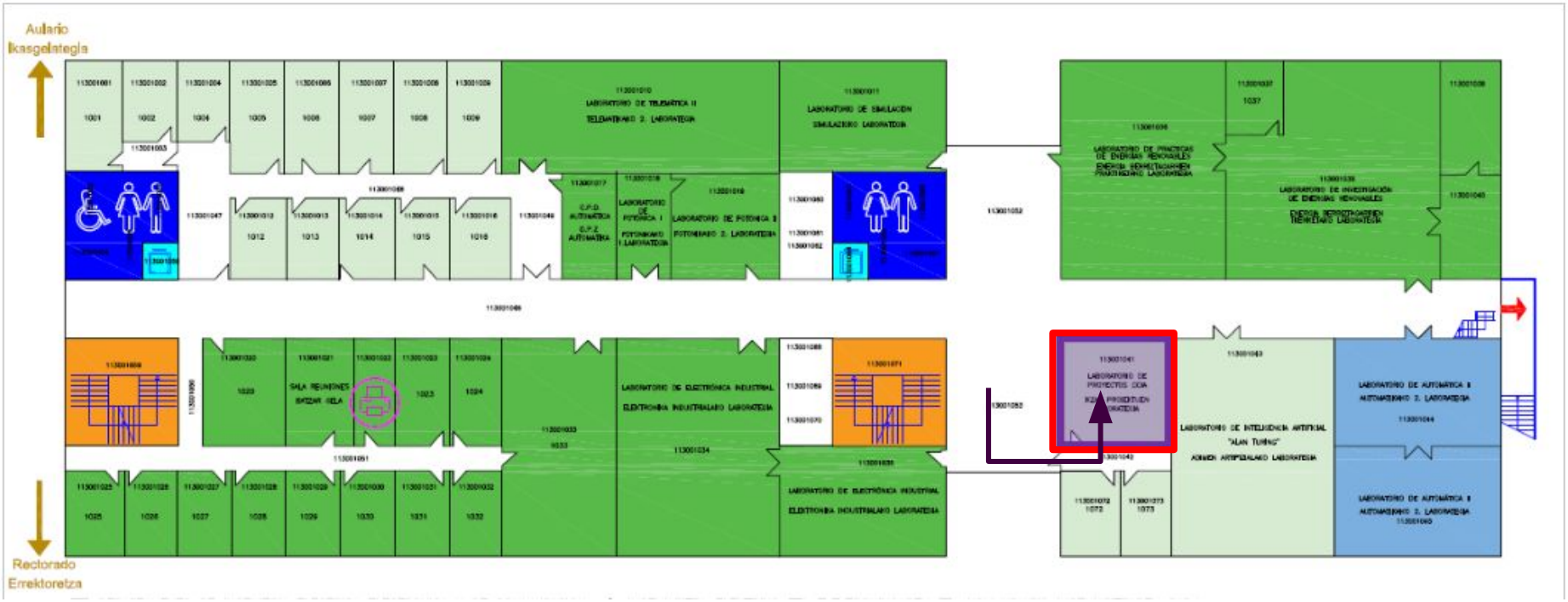
Iris Domínguez / Augusto Mauch

Curso 2026/2027

¿Quién soy yo?

- Augusto Mauch Goya (augusto.mauch@unavarra.es)
- Licenciado en Ingeniería en Informática (UPNA)
- Profesor asociado (6 años)
  - 4 años en diferentes departamentos de la UPNA
  - MTorres
  - Orisha-Openbravo (~15 años)

Despacho Augusto, Los Pinos, 1ª planta



Horario de tutorías por decidir, confirmaré la semana que viene

Hay que reservar las tutorías desde MiAulario (apartado Citas)

Calendario - Prácticas

- P2-1: Viernes 15h-17h Aula A-329
- P2-2: Viernes 17h-19h Aula A-329

- División de grupos publicada en MiAulario
- Cambios de grupo: consultar con el profesor dependerá de la disponbilidad de espacio

Contenido de las prácticas

Primera parte, implementación y uso de estructuras de datos

Segunda parte, elaboración de un trabajo en grupo

Objetivo:

- Profundizar conocimiento de estructuras de datos
- Puesta en práctica de técnicas de algoritmos vistas en clase
- Profundizar conocimiento de C
- Mejorar el trabajo en equipo y la presentacioń de resultados

Contenido de las prácticas

- **Bloque #1: Estructuras de datos**
  - A) Semanas 1-3: Pilas, colas, listas
  - B) Semanas 4-6: Árboles y montículos
  - Semana 7: Simulacro de examen, examen
- **Bloque #2: Trabajo**
  - Semana 8: Introducción al trabajo
  - Semanas 9-13: Trabajo en grupos
  - Semana 13: Presentación del trabajo

Septiembre	11	Bloque #1.A
	18	Bloque #1.A
	25	Bloque #1.A
Octubre	2	Bloque #1.B
	9	Bloque #1.B
	16	Bloque #1.B
	23	Simulacro de Examen
	30	Trabajo
Noviembre	6	Trabajo
	13	Trabajo
	20	Trabajo
	27	Trabajo
Diciembre	4	Trabajo
	11	Presentación del trabajo

Criterios de evaluación

Sistema de evaluación	Peso	Recuperable
1-Examen teórico	45%	Sí
2-Examen práctico	**15%**	**Sí**
3-Trabajo de prácticas	**20%**	**No**
4-Evaluación contínua	20%	No

- Para aprobar la asignatura:
  - **Examen práctico >= 4**
  - **Media ponderada >= 5**
  - Si Evaluación continua >= 6 -> Examen teórico >= 4.5
  - Si Evaluación continua < 6 -> Examen teórico >= 5

Calendario - Exámenes

- **Jueves 29 de Octubre, 12:00-14:00**

  Examen práctico
- **Viernes 11 de Diciembre, horario de clase**

  Presentación de trabajos
- **Lunes 18 de Enero, horario por decidir**

  Examen de recuperación de prácticas

Impacto de la IA Generativa

El trabajo de desarrollador de Software ha cambiado, y va a seguir cambiando.

Conocer los fundamentos del desarrollo sigue siendo importantísimo, pero ahora también lo es saber sacar provecho a las herramientas de IA Generativa (ChatGPT, Claude, Copilot, etc):

- NO para que hagan nuestro trabajo
- SÍ para que nos ayuden a entender los fundamentos, a ir más allá

Impacto de la IA Generativa

Aplicado a esta asignatura:

- Usad esas herramientas para resolver dudas (contad conmigo siemper en clase y tutorías), profundizar más allá de lo que podríamos sin ellas
- No les pidáis que haga vuestro trabajo (i.e. “lee este enunciado e implementa una pila en c”)

El examen práctico lo haréis sin apuntes y sin acceso a internet, no lleguéis a depender de esas herramientas

En la presentación del trabajo cada equipo tendrá que defender su trabajo - si ChatGPT hace el trabajo, se notará en la defensa

Bloque #1: Estructuras de datos

Dos tipos de ejercicios:

A. Implementación de una estructura de datos
B. Problema que se puede resolver utilizando algunas de esas estructuras de datos

En el examen de prácticas la estructura será la misma.

A) Implementación de una estructura de datos

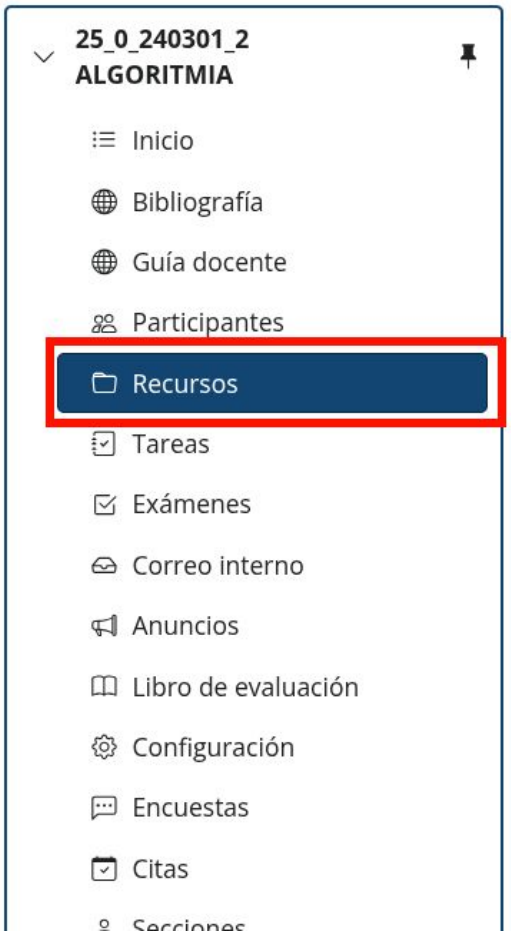
Incluímos:

- **Enunciado**
- **Fichero <Estructura>.h**
- **Fichero ejecuta<Estructura>.c**

Vosotros tenéis que las funciones definidas en <Estructura>.h en el fichero <Estructura>.c

Bloque #1: Estructuras de datos

Los enunciados ya está en MiAulario:





Repaso

Punteros

Reserva y liberación dinámica de memoria

. vs ->

Compilación

Punteros

```
int x = 10;
int *ptr = &x;
int valor = *ptr;
```

variable	posición	**valor**
x	f0x001	**10**
ptr	f0x002	**f0x001**
valor	f0x003	**10**

"memoria"

Repaso: reserva dinámica de memoria

- **malloc**: recibe un parámetro, número de bytes a reservar. sizeof devuelve el número de bytes de la estructura que se le pase como parámetro

```
void apilar(tipoPila *p, tipoElementoPila elemento){
    celdaPila *nuevo;
    nuevo=(celdaPila*)malloc(sizeof(celdaPila));
    nuevo->elem=elemento;
    nuevo->sig=*p;
    *p=nuevo;
}
```

```
void encolar(tipoCola* cola, tipoElementoCola elemento){
    celdaCola* nueva_celda;
    nueva_celda = (celdaCola*)malloc(sizeof(celdaCola));
    nueva_celda->elem = elemento;
    nueva_celda->sig = NULL;
    ...
```

Repaso: liberación dinámica de memoria

- **free**: libera el espacio de memoria ocupado por la estructura a la que apunta un puntero

```
void desapilar(tipoPila *p){
    if (esNulaPila(*p))
        errorPila("No se puede desapilar en una pila vacia.");
    else{
        celdaPila *aux;
        aux=*p;
        *p=(*p)->sig;
        free(aux);
    }
}
```

```
void desencolar(tipoCola* cola){
    if (esNulaCola(*cola)){
        printf("Error: No se puede desencolar de cola nula");
    }else{
        celdaCola* aux_celda;
        aux_celda = cola->ini;
        cola->ini = (cola->ini)->sig;
        free(aux_celda);
        if (esNulaCola(*cola)){
            cola->fin = NULL;
        }
    }
}
```

Cuando usar . vs ->

Si la variable es un puntero a una estructura, variable->propiedad

```
typedef struct tipoC{
    celdaCola* ini;
    celdaCola* fin;
}tipoCola;
```

```
void nuevaCola(tipoCola* cola){
    cola->ini = NULL;
    cola->fin = NULL;
    }
```

Si la variable es una estructura, variable.propiedad

```
bool esNulaCola(tipoCola cola){
    return cola.ini == NULL;
}
```

Pregunta, ¿Porqué en este caso usamos variable.propiedad?

```
typedef struct celdaP{
    tipoElementoPila elem;
    struct celdaP* sig;
}celdaPila;
typedef celdaPila* tipoPila;
```

```
tipoElemento cima(tipoPila p){
    if (esNulaPila(p)){
        exit(-1);
    }else{
            return p->elem;
    }
};
```

Cuándo usar . vs ->

Lo siguiente es equivalente

```
tipoElementoPila cima(tipoPila p){
    if (esNulaPila(p))
        …
    else
        return p->elem;
};
```

```
tipoElementoPila cima(tipoPila p){
    if (esNulaPila(p))
        …
    else
        return (p*).elem;
};
```

Si p es un puntero a una celda, (p*) representa la celda

Cuándo salir con error - exit(-1)

Recordatorio: Siempre salir con error si es imposible ejecutar una función si la estructura de datos cumple con determinadas condiciones, sobre todo si la función devuelve algo que no podemos obtener.

```
tipoElementoPila cima(tipoPila p){
     if (esNulaPila(p)){
          printf("\nERROR: No existe cima de la pila NULA\n");
          exit(-1);
     }else{
          return p->elem;
     }
};
```

```
tipoElementoCola frente(tipoCola cola){
     if (esNulaPila(cola)){
          printf("\nERROR: No existe frente de la cola NULA\n");
          exit(-1);
     }else{
          return (cola.ini)->elem;
     }
};
```

Repaso: compilación

Entender/memorizar la compilación (sólo dos casos, librería o ejecutable)

```
gcc -c -Wall <Estructura>.c
gcc -Wall ejecuta<Estructura>.c <Estructura>.o
```

Compilar siempre con la opción -Wall: los warnings siempre mostrarán problemas reales

Compilar muy a menudo, después de cada cambio que hagáis / cada función que implementéis

Repaso: compilación

Compilar y ejecutar

```
gcc -c -Wall <Estructura>.c
gcc -Wall ejecuta<Estructura>.c <Estructura>.o &&
./a.out
```

de una vez

```
gcc -Wall ejecuta<Estructura>.c <Estructura>.c &&
./a.out
```

Renombrar

```
gcc -Wall ejecuta<Estructura>.c <Estructura>.c -o
ejecuta<Estructura>
```

Repaso de la pila

ALGORITMIA · SEGUNDO CURSO

Pila dinámica, paso a paso

Dibuja primero, programa después. Cada línea de código, al lado del dibujo que le corresponde.

LA ESTRUCTURA

Práctica 1 · P1.1 · pilaEnterosDinamica

```
typedef int tipoElementoPila;

typedef struct celdaP{
    tipoElementoPila elem;
    struct celdaP *sig;
} celdaPila;

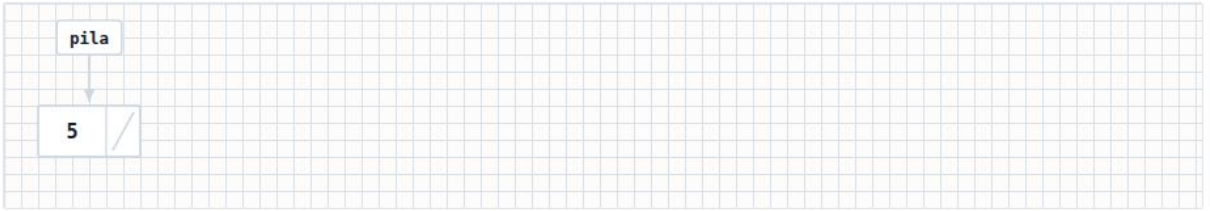
typedef celdaPila* tipoPila;
```

Una pila **es** un puntero: `tipoPila` no es más que la dirección de la celda que está en la cima. Cada celda guarda su valor y la dirección de la siguiente. La última guarda `NULL`, que es lo que marca el final; en los dibujos se ve como una **barra diagonal** en el campo `sig`.

pila == NULL — no hay ninguna celda. Ojo: la pila existe como variable, lo que no existe es ninguna celda.



pila → 5 — una sola celda, y su campo sig vale NULL.



pila → 7, 5, 3 — el 7 es la cima. Las celdas no están una al lado de otra en memoria: lo único que las une son las flechas.

